BUILD SKILLS THAT LAST Unlock every paid feature

JavaScript Generators

Previous Next

A JavaScript Function can only return **one value**.

A JavaScript Generator can **return multiple values**, one by one.

A JavaScript Generator can yield a **stream of data**.

A JavaScript Generator can be **paused** and **resumed**.

Generator Functions

A Generator Function is defined using the **function*** syntax:

```
function* myGenerator() {}
```

When called, a generator function returns a **Generator Object**, not a direct value:

```
{value:1, done:false}
```

Generator Objects

A **Generator Object** is returned by the generator function.

It conforms to both the iterable and iterator protocols, meaning it can be iterated over using **for...of loops**.

```
function* myStream() {
// return {value:1, done:false}
yield 1;

// return {value:2, done:false}
yield 2;

// return {value:3, done:true}
return 3;
}

// Create a Generator
let myGenerator = myStream();

// Iterate over the Generator
for (let value of myGenerator) { // code }
```

Try it Yourself »

The yield Keyword

The **yield** keyword pauses execution and **yield a value** back to the caller.

The generator's state is preserved, and can be **resumed** from the point of the yield when **next()** is called.

Generators maintain their internal state between yield calls, allowing them to continue execution from where they left off.

Note

The code example above returns 1 and 2. Not 3!

The for..of loop exits when **done:true**.

To get all the values, return them with yield.

Example

```
// return {value:2, done:false}
yield 2;

// return {value:3, done:false}
yeald 3;
}

// Create a Generator
let myGenerator = myStream();

// Iterate over the Generator
for (let value of myGenerator) { // code }
```

Try it Yourself »

Generator Object Methods

A Generator Object conforms to the **iterable and iterator protocols**.

It can be iterated over with **for...of loops**.

It also has **methods**:

Method	Description
next()	Resumes the next execution of a generator function
return()	Finishes this execution and returns a given value
throw()	Throws an exeption in this execution

The next() Method

When called on the Generator object it **resumes** execution of the generator function.

It returns an object with and value and a boolean, indicating if the generator has finished:

{ **value**: *value*, **done**: `true|false` }

They can efficiently generate values on demand, making them suitable for potentially **infinite data streams**.

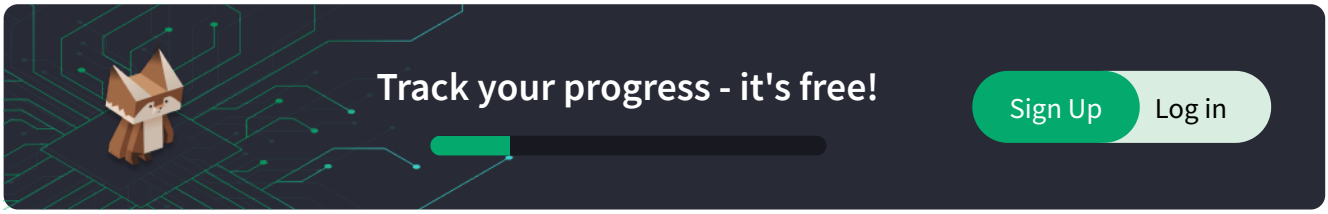
They can **pause and resume** execution, which can be useful for fine-grained control over program flow.

Before async/await, generators were used with Promises to manage asynchronous operations in a more sequential-looking manner.

Note

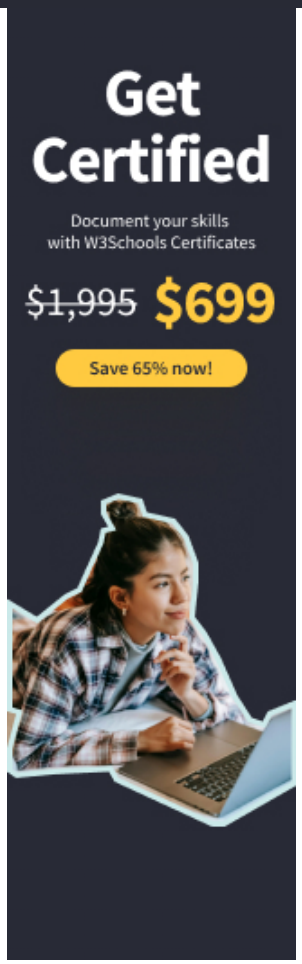
Generators bring the **iteration concept** directly into the core JavaScript language and provide a mechanism for customizing the behavior of for...of.

❮ Previous Next ❯

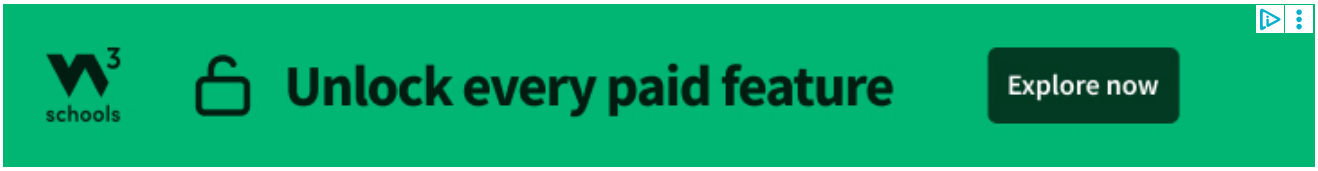


ADVERTISEMENT

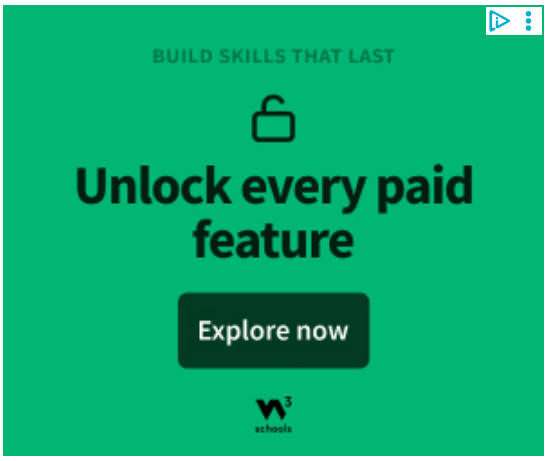
COLOR PICKER



ADVERTISEMENT



ADVERTISEMENT



Top Tutorials

HTML Tutorial
CSS Tutorial
JavaScript Tutorial
How To Tutorial
SQL Tutorial
Python Tutorial
W3.CSS Tutorial
Bootstrap Tutorial
PHP Tutorial
Java Tutorial
C++ Tutorial
jQuery Tutorial

Top References

HTML Reference
CSS Reference
JavaScript Reference
SQL Reference
Python Reference
W3.CSS Reference
Bootstrap Reference
PHP Reference
HTML Colors
Java Reference
Angular Reference
jQuery Reference

Top Examples

HTML Examples
CSS Examples
JavaScript Examples
How To Examples
SQL Examples
Python Examples
W3.CSS Examples
Bootstrap Examples
PHP Examples
Java Examples
XML Examples
jQuery Examples

Get Certified

HTML Certificate
CSS Certificate
JavaScript Certificate
Front End Certificate
SQL Certificate
Python Certificate
PHP Certificate
jQuery Certificate
Java Certificate
C++ Certificate
C# Certificate
XML Certificate

Copyright 1999-2025 by Refsnes Data. All Rights Reserved. W3Schools is Powered by W3.CSS.